

Here is the complete, step-by-step guide to building this Django Student Management project from scratch. Follow these steps exactly to set up your environment, write the code, and launch your site.

Step 1: Set Up Your Project Folder & Environment

Open your computer's terminal (or command prompt) and run these commands to build your workspace.

1. Create a folder for your project and enter it

```
mkdir student_project  
cd student_project
```

2. Create a virtual environment (keeps dependencies clean)

```
python -m venv venv
```

3. Activate the virtual environment

On Windows:

```
venv\Scripts\activate
```

On Mac/Linux:

```
source venv/bin/activate
```

4. Install Django

```
pip install django
```

Step 2: Initialize Django Project and App

Create the master project configurations and the application structure.

1. Start the main project structure named 'config'

```
django-admin startproject config .
```

2. Create the application component for students

```
python manage.py startapp students
```

Step 3: Register Your App (config/settings.py)

Open the config/settings.py file. Scroll down to the INSTALLED_APPS section and add 'students', at the bottom of the list.

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.sessions',  
    'django.messages',  
    'django.contrib.staticfiles',  
    'students', # Add this line  
]
```

Step 4: Create the Database Model (students/models.py)

Open students/models.py and replace its contents with the code below to define the structure of the SQLite database table.

```
from django.db import models  
  
class Student(models.Model):  
    name = models.CharField(max_length=100)  
    email = models.EmailField(unique=True)  
    age = models.IntegerField()  
    course = models.CharField(max_length=100)  
    marks = models.FloatField()  
  
    def __str__(self):  
        return self.name
```

Step 5: Connect URL Routing

We need to tell Django how to direct browser requests to our application.

1. Update the Main Project Routes (config/urls.py):

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('students/', include('students.urls')), # Points to your app's URLs
]
```

2. Create App-Specific Routes (students/urls.py):

Create a new file named urls.py inside your students directory and paste this code:

```
from django.urls import path
from . import views

urlpatterns = [
    path("", views.home, name='home'),
    path('list/', views.student_list, name='list'),
    path('<int:id>/', views.student_detail, name='detail'),
]
```

Step 6: Write Logic inside Views (students/views.py)

Open students/views.py and implement the logic using Django's ORM to fetch data from SQLite and serve it to the frontend templates.

```
from django.shortcuts import render
from .models import Student

# Home Dashboard Page View
def home(request):
    return render(request, 'students/home.html')

# Fetch and Render All Records View
def student_list(request):
    students_dataset = Student.objects.all()
    return render(request, 'students/student_list.html', {'students': students_dataset})

# Fetch and Render a Specific Profile View
def student_detail(request, id):
```

```
single_student = Student.objects.get(id=id)
return render(request, 'students/student_detail.html', {'student': single_student})
```

Step 7: Create HTML Templates

You need to match the specific folder naming conventions Django requires for automatic template loading.

1. Inside your students app directory, create a folder named templates.
2. Inside templates, create another folder named students.
3. Create your three HTML files inside that nested students/templates/students/ path.

1. home.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>Dashboard</title>
  <link rel="stylesheet" href="{% static 'students/css/style.css' %}">
</head>
<body>
  <h1>Welcome to the Student Management System</h1>
  <a href="{% url 'list' %}">Go to Student Roster</a>
</body>
</html>
```

2. student_list.html

```
{% load static %}
<!DOCTYPE html>
<html>
<head>
  <title>All Students</title>
  <link rel="stylesheet" href="{% static 'students/css/style.css' %}">
</head>
<body>
  <h1>Registered Students</h1>
  <table border="1" cellpadding="10" cellspacing="0">
    <thead>
```

```

        <tr>
            <th>Name</th>
            <th>Course</th>
            <th>Actions</th>
        </tr>
    </thead>
    <tbody>
        {% for student in students %}
        <tr>
            <td>{{ student.name }}</td>
            <td>{{ student.course }}</td>
            <td><a href="{% url 'detail' student.id %}">View Profile</a></td>
        </tr>
        {% empty %}
        <tr><td colspan="3">No students found.</td></tr>
        {% endfor %}
    </tbody>
</table>
</body>
</html>

```

3. student_detail.html

```

{% load static %}
<!DOCTYPE html>
<html>
<head>
    <title>Profile - {{ student.name }}</title>
    <link rel="stylesheet" href="{% static 'students/css/style.css' %}">
</head>
<body>
    <h1>Student Details</h1>
    <div style="border: 1px solid #ddd; padding: 20px; width: 300px;">
        <p><strong>ID:</strong> {{ student.id }}</p>
        <p><strong>Name:</strong> {{ student.name }}</p>
        <p><strong>Email:</strong> {{ student.email }}</p>
        <p><strong>Age:</strong> {{ student.age }}</p>
        <p><strong>Course:</strong> {{ student.course }}</p>
        <p><strong>Marks:</strong> {{ student.marks }}%</p>
    </div>
    <br>
    <a href="{% url 'list' %}">Back to Roster</a>
</body>
</html>

```

Step 8: Build the Database & Run the Project

Execute the setup operations in your terminal to build out your local SQLite file (db.sqlite3).

```
# 1. Look for changes in models.py and generate blueprint files
python manage.py makemigrations
```

```
# 2. Execute blueprints to construct the actual SQL tables
python manage.py migrate
```

```
# 3. Open up the Django database python shell to input data
python manage.py shell
```

Inside the open interactive python shell prompt (>>>), run this block to inject dummy records:

```
from students.models import Student
Student.objects.create(name="Alice Smith", email="alice@test.com", age=20,
course="Python", marks=88.5)
Student.objects.create(name="Bob Jones", email="bob@test.com", age=22, course="Web
Dev", marks=75.0)
exit()
```

Finally, start the local engine server:

```
python manage.py runserver
```

Open up your web browser and navigate to <http://127.0.0.1> to see your system running with live database information!

Would you like to know how to create a **form** so users can **add new students** directly from the website without using the terminal shell?